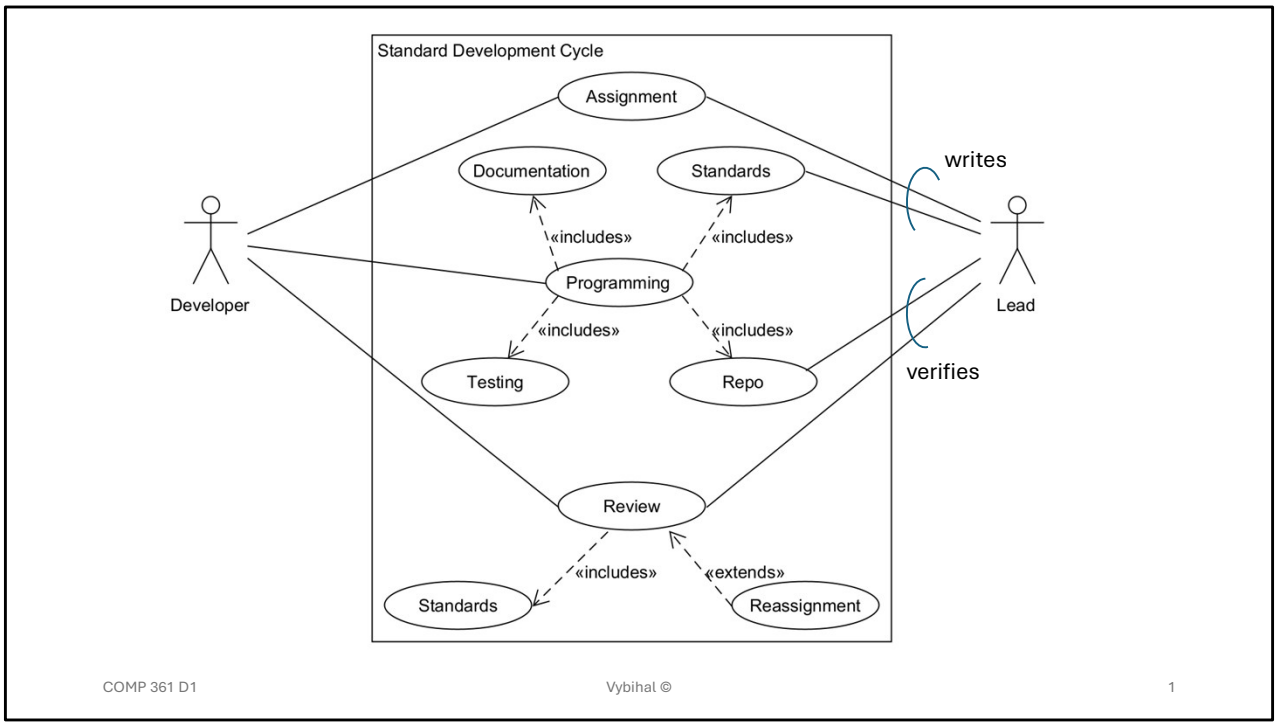
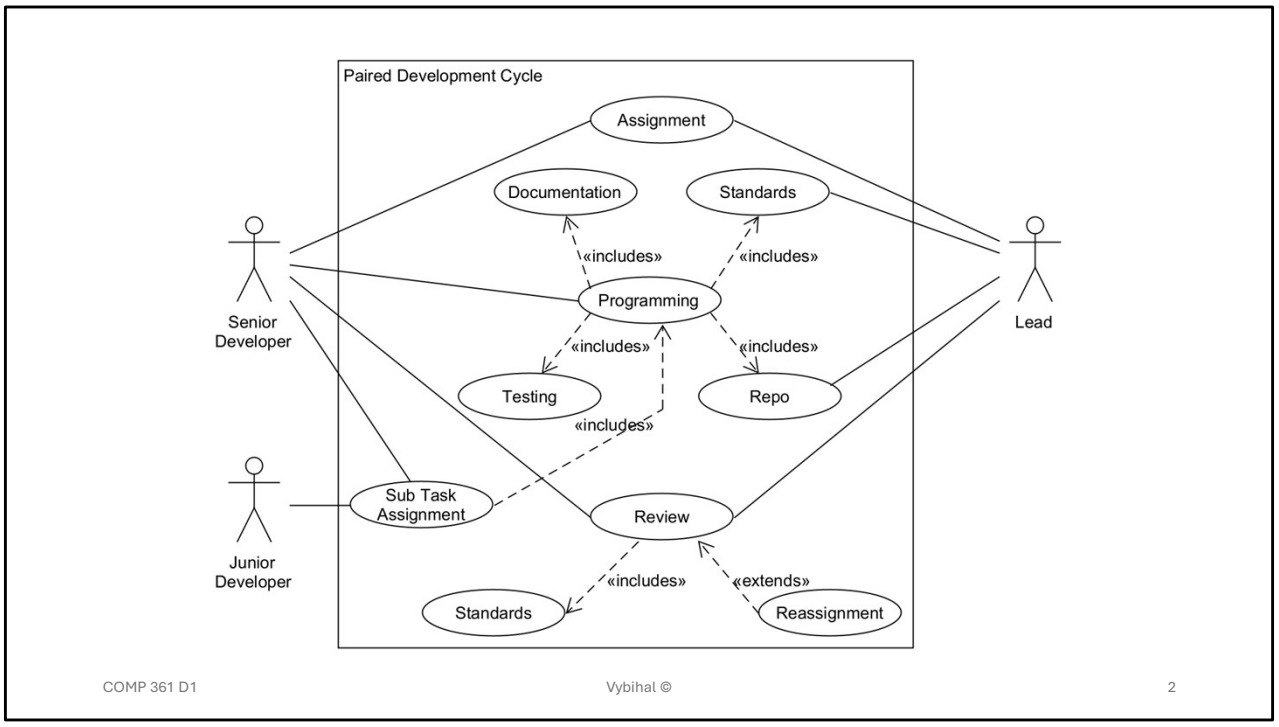
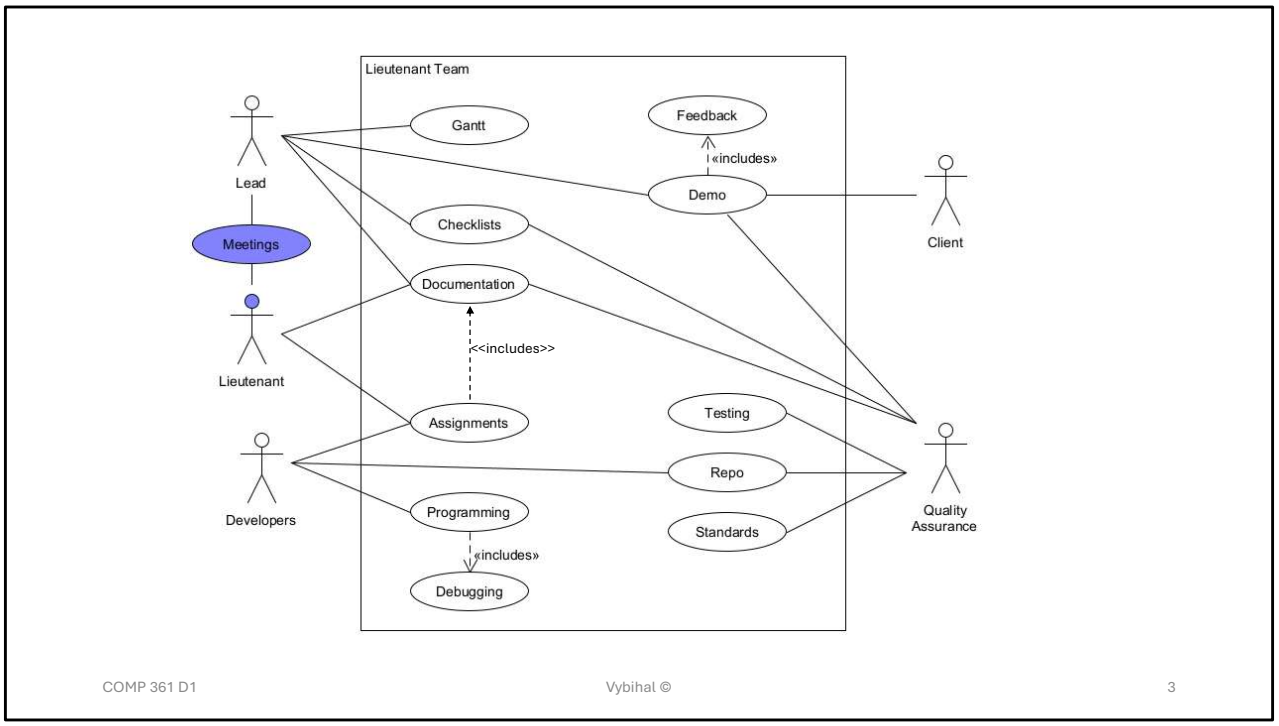


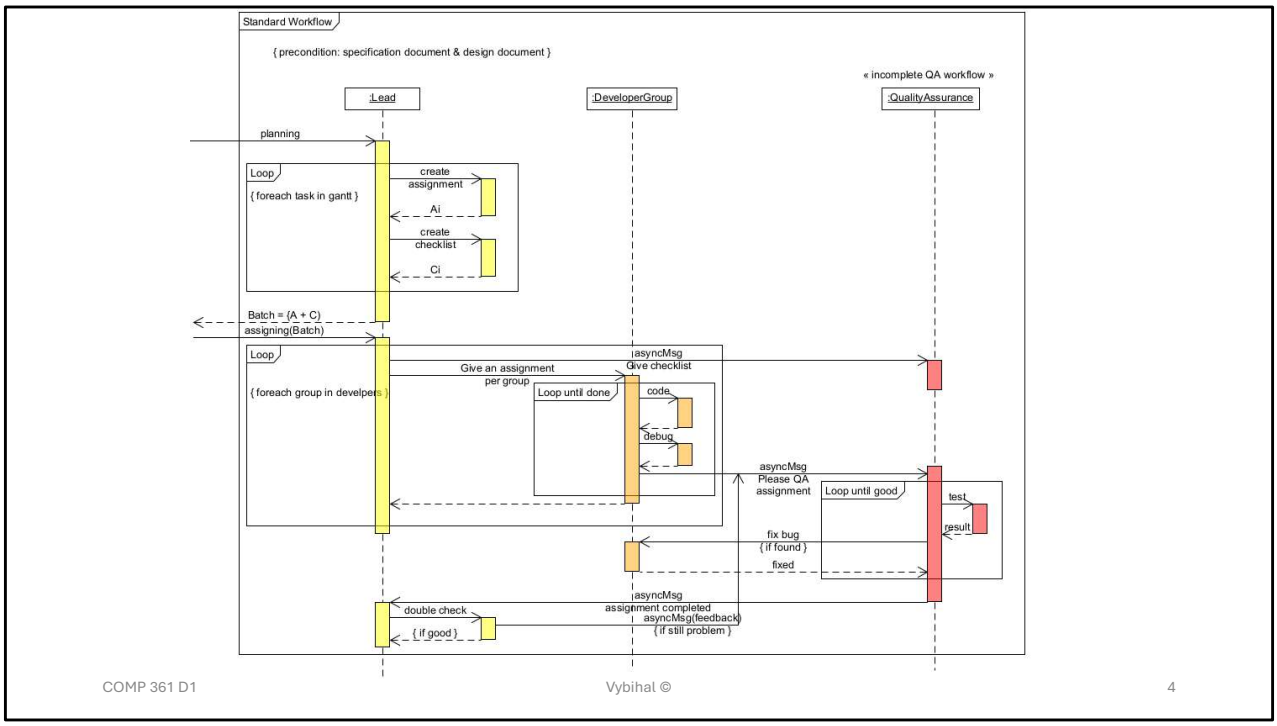
In the standard development cycle, two roles exist: the Lead and the Developer. The Lead's job is threefold: plan the project, assign work to the developers, and ensure the work is completed as requested. The Use Case diagram above attempts to define the tasks each person is involved in. Starting with the Lead, you see their job is divided between writing and reviewing. The Lead will create work assignments for each developer that follow industry standards and the specification documentation created with the client. Then the Lead verifies the work by inspecting and curating the repository and setting up demos with each developer to review the source code and application runtime. Developers build software. They receive assignments and try to complete their tasks by the assignment's due date. Developers commit and merge their code into a repository, updating the planner boards or progress forms to document their progress. Finally, they make appointments with the Lead to review the work they completed. The Lead documents issues and improvement requests, then gives them to the Developer to incorporate into their code. Once the review is complete and there are no issues, the Lead gives the Developer the next assignment.



In a Paired development cycle, two developers are assigned the same assignment by the Lead. The cycle behaves the same way as the Standard cycle, except that there are two developers. The above Use Case assumes the pair is based on seniority, but the pair can be based on other requirements. For example, the assignment is hard, and two experts are needed to complete it. Or the assignment requires two different skills, so developers with the corresponding skills are paired together. Or you simply want to complete the assignment more quickly. When the Lead gives the assignment to the Pair, the immediate task is to divide the work into two. If it is a Junior and Senior pair, the Senior divides the work, and the Junior reports back to the Senior. The Senior directly communicates with the Lead.



In larger projects, the Lead has too much work. In the workflow above, the Lead has delegated the job to a Lieutenant and the Quality Assurance person. In larger projects, the Quality Assurance person can be a team with its own lead. In these large projects, the above Use Case would change the name from Quality Assurance to Quality Assurance Lead. Notice that the Lead still writes the documentation and communicates with the client, but their primary task after creating the documentation is Project Tracking and scheduling using the Gantt chart and the Checklists. Assignment writing has been delegated to the Lieutenant who maintains regular communication with the Lead. The Quality Assurance person is responsible for the curation of the repository, validating the software, and demoing with the client. The Lead is also involved with the client demonstrations and is in control over the things QA will be checking through the Checklists. The Lead writes Checklists for each assignment requesting for specific testing mechanisms. These Checklists are returned to the Lead as a form of progress reporting after QA has reviewed a developer's submission.

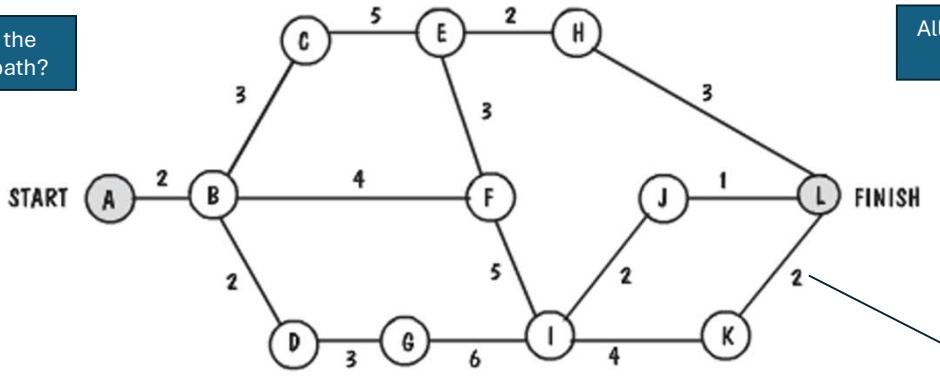


This flowchart shows what a standard team workflow looks like. It begins with the Lead. For each task in the Backlog, the Lead creates an Assignment and Checklist. Each Assignment and Checklist is assigned to a Developer and a Quality Assurance person. The QA person waits for the Developer to finish programming and testing locally. Then the Developer sends the completed code to the QA person to verify according to the Checklist. The QA Person writes notes on the Checklist documenting what worked and what didn't, and suggests improvements. This is returned to both the Developer and the Lead (or Lieutenant). The Developer makes the corrections, possibly meeting with the Lead or Lieutenant to discuss them. Once that is done, the Developer sends version 2 of their code to the QA person, and we repeat the process until the Assignment is complete. At the very end, the completed code is shown to the client for their final approval. If that okay is received, then the Developer can be given the next Assignment; otherwise, the process is reopened with the Client's issues.

Longest Covering Path to Completion: Activity Graph

What is the longest path?

All nodes must complete

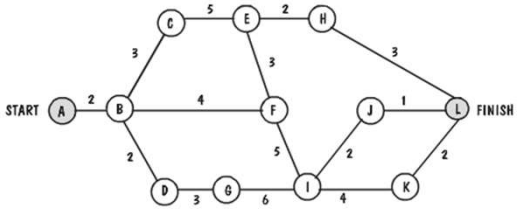


- Activity graphs can be constructed to focus only on the app alone or cover the entire development process.
- Optimization occurs by maximizing parallelism & shortening arc lengths.
- Node = either a module of the app or a phase is the development process.
- Arc = the number of hours/days it takes to complete a Node.
- This is not a cyclic graph: longest path is the earliest project can complete.

Node K takes 2 days to complete

An Activity Graph organizes all the tasks from the Backlog into an ordered list of activities, including parallel development. A project's ability to be developed in parallel lies in two qualities: (a) the natural tasks within the project that can be done in parallel, and (b) the number of team members you have. The graph has a starting point and an ending point. Between these points are the number of activities that can be carried out in parallel, represented as arcs. Within each arc are the tasks that cannot be parallelized, ordered in the sequence they need to be completed. Each task is in a circle. The arc connecting two circles is labeled with the number of days it will take to transition between the two circles. Arc merge points are where two parallel arcs meet. The longest path through the graph is the project duration, assuming these parallel paths can be executed. It is important to keep in mind: (1) you may have more team members than parallel paths, or (2) there may be more parallel paths than team members. This is important. It means that the Activity Graph you build must be constrained by the weakest link. For example, if you have more parallel arcs than team members, you need to reduce the number of parallel arcs to match the number of team members. Also note that if you are working in a Paired workflow, then you have a reduced capacity to handle parallel arcs.

Effort Estimation



- Total means the number of days or hours someone needs to work to complete the work of every node. The hours we need to pay someone.
- Optimal means the number of calendar days or hours to complete the project with an infinite number of people.
- Practical means the number of calendar days or hours with the given constraints.

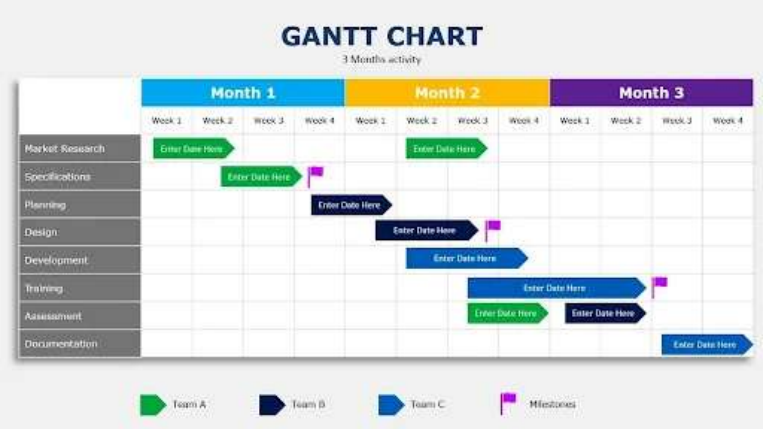
Node	Duration	Pre-conditions
A – B	2	Met all stakeholders
B – C	3	Docker image available
B – F	4	Internet available

Total 47
Optimal (need optimal graph)
Practical (need practical graph)

The table on the side helps you document all the durations and what they mean.

Gantt Chart

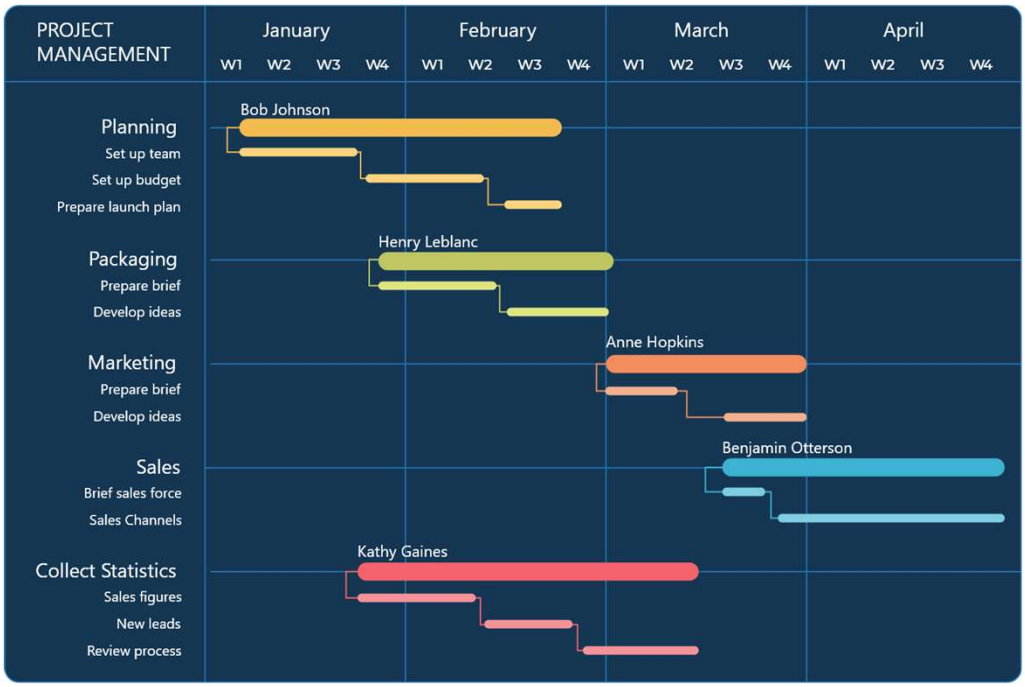
If we work only Monday to Friday, no vacations or holidays, how many calendar workdays is 47 days?



- Based on actual calendar
  - Our 47 days includes weekend, holidays, and vacations.
- Rows
  - Project phases, and
  - Software modules
- Columns
  - Actual start & end dates
- Flags
  - Important milestones
  - $ penalties may apply
- Allows for parallelism
- Allows for teams
- Know where you are/should be

Great for the team leader and the client to track the state of the project.

Once your Activity Graph is complete, you must place it on a Calendar. The Activity Graph just counts the number of days between tasks. It doesn't account for weekends, holidays, or sick days. The Gantt Chart asks you to map the Activity Graph onto a calendar while respecting the regular workweek and expected holidays, etc. It is useful from the Lead's and Client's point of view because, on any given calendar date, they can look at the chart to see what should have been completed. For example, find the date in the top row of the chart, then draw a vertical line from the date to the bottom of the page; everything to the left of that line should have been completed. This chart can be color-coded as needed, for example, Frontend in blue and Backend in yellow. Add flags to highlight important due dates, such as milestones or key demonstrations.



There are many Gantt Chart formats. In this graph, the person responsible is identified both by color-coding and by writing their name at the beginning. Some other Gantt charts use a very wide left margin to record key data points for each task. You can Google to see additional examples. As a rule, simplicity is better when charting, but seek balance, because too simple becomes useless.